

Lec 6

The back-propagation

Dr. Mohammad Mhawish

- The back-propagation learning algorithm is one of the most important developments in neural networks
- This learning algorithm is applied to multilayer feed-forward networks consisting of processing elements with continuous differentiable activation functions. The networks associated with back-propagation learning algorithm are also called *back-propagation networks* (BPNs).
- For a given set of training input-output pair, this algorithm provides a procedure for changing the weights in a BPN to classify the given input patterns correctly.
- The basic concept for this weight update algorithm is simply the gradient-descent method as used in the case of simple perceptron networks with differentiable units.
- This is a method where the error is propagated back to the hidden unit.

- The back-propagation algorithm is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
- The general difficulty with the multilayer perceptrons is calculating the weights of the hidden layers in an efficient way that would result in a very small or zero output error.
- When the hidden layers are increased the network training becomes more complex. To update weights, the error must be calculated.
- The error, which is the difference between the actual (calculated) and the desired (target) output, is easily measured at the output layer.
- It should be noted that at the hidden layers, there is no direct information of the error. Therefore, other techniques should be used to calculate an error at the hidden layer, which will cause minimization of the output error, and this is the ultimate goal.

- The training of the BPN is done in three stages
 - the feed-forward of the input training pattern
 - the calculation and back-propagation of the error,
 - and updation of weights.
- The testing of the BPN involves the computation of feed-forward phase only. There can be more than one hidden layer (more beneficial) but one hidden layer is sufficient. Even though the training is very slow, once the network is trained it can produce its outputs very rapidly.

Activation function

- The activation function employed in a backpropagation net must satisfy certain properties.
- It must be continuous, differentiable, and monotonically non-decreasing. Moreover, it is desirable the first derivative of the activation function be easily computable.

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

The derivative of this function is

$$f'(x) = \lambda f(x)[1 - f(x)]$$

$$f(x) = \frac{2}{1 + e^{-\lambda x}} - 1 = \frac{1 - e^{-\lambda x}}{1 + e^{-\lambda x}}$$

The derivative of this function is

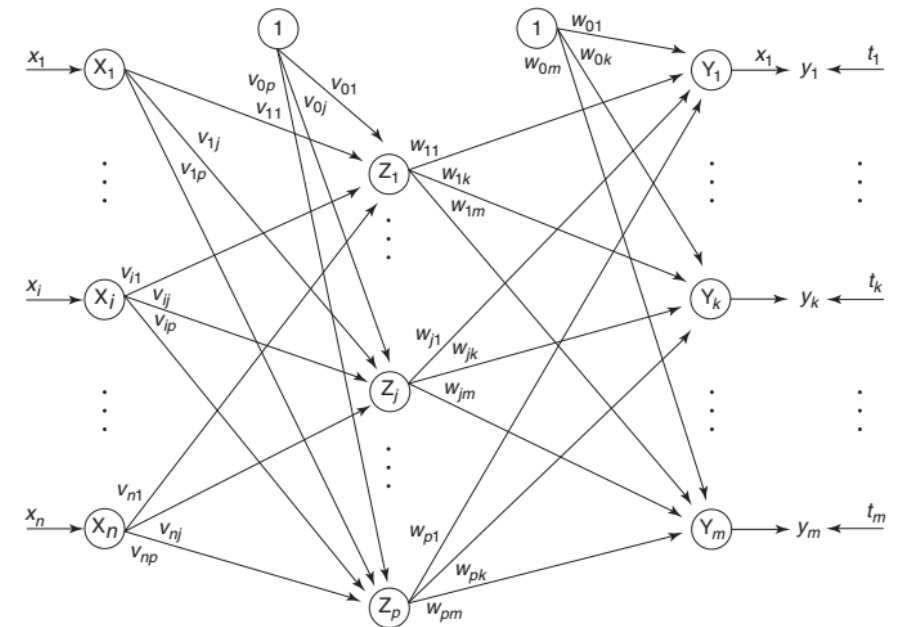
$$f'(x) = \frac{\lambda}{2} [1 + f(x)][1 - f(x)]$$

Architecture

- A back-propagation neural network is a multilayer, feed-forward neural network consisting of an input layer, a hidden layer and an output layer.
- The neurons present in the hidden and output layers have biases, which are the connections from the units whose activation is always 1. The bias terms also acts as weights.
- Figure shows the architecture of a BPN, depicting only the direction of information flow for the feed-forward phase. During the back-propagation phase of learning, signals are sent in the reverse direction.

The inputs are sent to the BPN and the output obtained from the net could be either binary (0, 1) or bipolar (−1, +1).

The activation function could be any function which increases monotonically and is also differentiable.



Derivation

- File ...
- Forward
- Backword

x = input training vector ($x_1, \dots, x_i, \dots, x_n$)

t = target output vector ($t_1, \dots, t_k, \dots, t_m$)

α = learning rate parameter

x_i = input unit i . (Since the input layer uses identity activation function, the input and output signals here are same.)

v_{0j} = bias on j th hidden unit

w_{0k} = bias on k th output unit

z_j = hidden unit j . The net input to z_j is

and the output is

$$z_j = f(z_{inj})$$

y_k = output unit k . The net input to y_k is

$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

and the output is

$$y_k = f(y_{ink})$$

δ_k = error correction weight adjustment for w_{jk} that is due to an error at output unit y_k , which is back-propagated to the hidden units that feed into unit y_k

δ_j = error correction weight adjustment for v_{ij} that is due to the back-propagation of error to the hidden unit z_j .

Also, it should be noted that the commonly used activation functions are binary sigmoidal and bipolar sigmoidal activation functions (discussed in Section 2.3.3). These functions are used in the BPN because of the following characteristics:

(i) continuity; (ii) differentiability; (iii) nondecreasing monotony.

The range of binary sigmoid is from 0 to 1, and for bipolar sigmoid it is from -1 to +1.

Training Algorithm

Step 0: Initialize weights and learning rate (take some small random values).

Step 1: Perform Steps 2–9 when stopping condition is false.

Step 2: Perform Steps 3–8 for each training pair.

Feed-forward phase (Phase I):

Step 3: Each input unit receives input signal x_i and sends it to the hidden unit ($i = 1$ to n).

Step 4: Each hidden unit z_j ($j = 1$ to p) sums its weighted input signals to calculate net input:

$$z_{inj} = v_{0j} + \sum_{i=1}^n x_i v_{ij}$$

Calculate output of the hidden unit by applying its activation functions over z_{inj} (binary or bipolar sigmoidal activation function):

$$z_j = f(z_{inj})$$

and send the output signal from the hidden unit to the input of output layer units.

Step 5: For each output unit y_k ($k = 1$ to m), calculate the net input:

$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

and apply the activation function to compute output signal

$$y_k = f(y_{ink})$$

Back-propagation of error (Phase II):

Step 6: Each output unit y_k ($k = 1$ to m) receives a target pattern corresponding to the input training pattern and computes the error correction term:

$$\delta_k = (t_k - y_k) f'(y_{ink})$$

The derivative $f'(y_{ink})$ can be calculated as in Section 2.3.3. On the basis of the calculated error correction term, update the change in weights and bias:

$$\Delta w_{jk} = \alpha \delta_k z_j; \quad \Delta w_{0k} = \alpha \delta_k$$

Also, send δ_k to the hidden layer backwards.

Step 7: Each hidden unit (z_j , $j = 1$ to p) sums its delta inputs from the output units:

$$\delta_{inj} = \sum_{k=1}^m \delta_k w_{jk}$$

The term δ_{inj} gets multiplied with the derivative of $f(z_{inj})$ to calculate the error term:

$$\delta_j = \delta_{inj} f'(z_{inj})$$

The derivative $f'(z_{inj})$ can be calculated as discussed in Section 2.3.3 depending on whether binary or bipolar sigmoidal function is used. On the basis of the calculated δ_j , update the change in weights and bias:

$$\Delta v_{ij} = \alpha \delta_j x_i; \quad \Delta v_{0j} = \alpha \delta_j$$

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

The derivative of this function is

$$f'(x) = \lambda f(x)[1 - f(x)]$$

Weight and bias updation (Phase III):

Step 8: Each output unit (y_k , $k = 1$ to m) updates the bias and weights:

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \Delta w_{jk}$$

$$w_{0k}(\text{new}) = w_{0k}(\text{old}) + \Delta w_{0k}$$

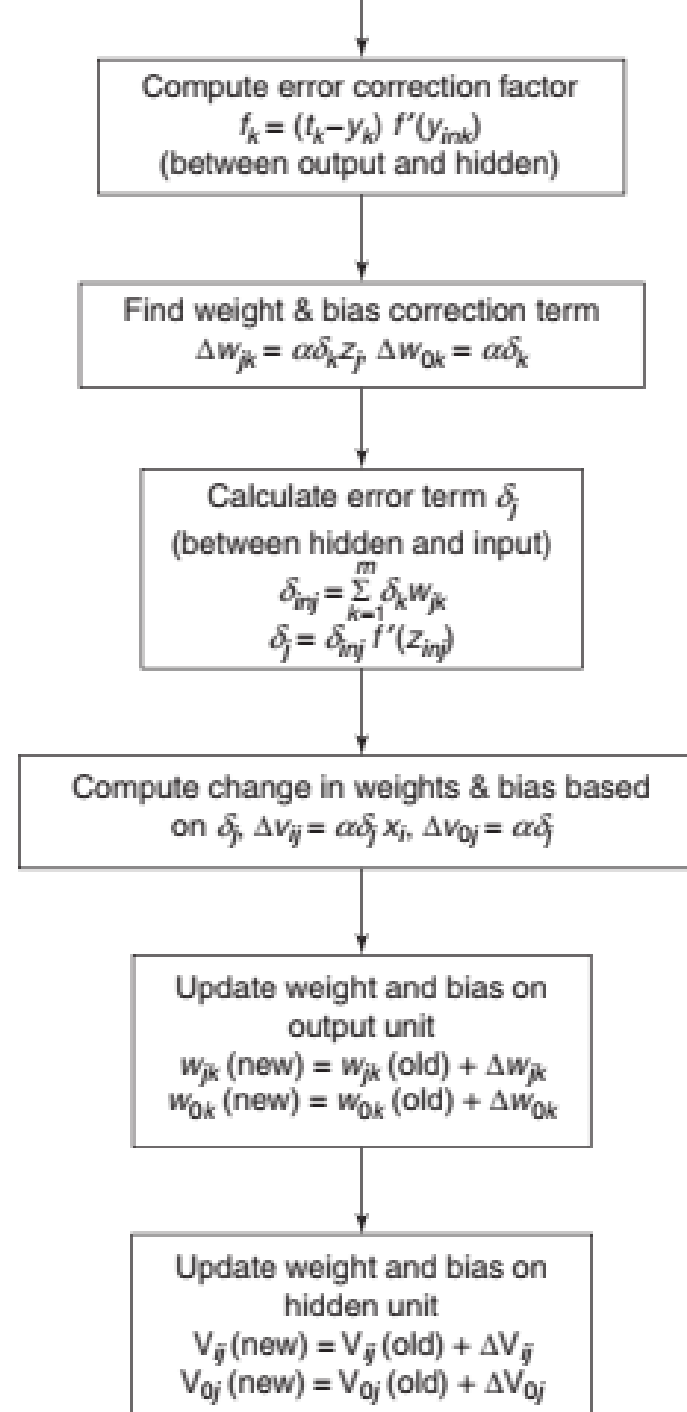
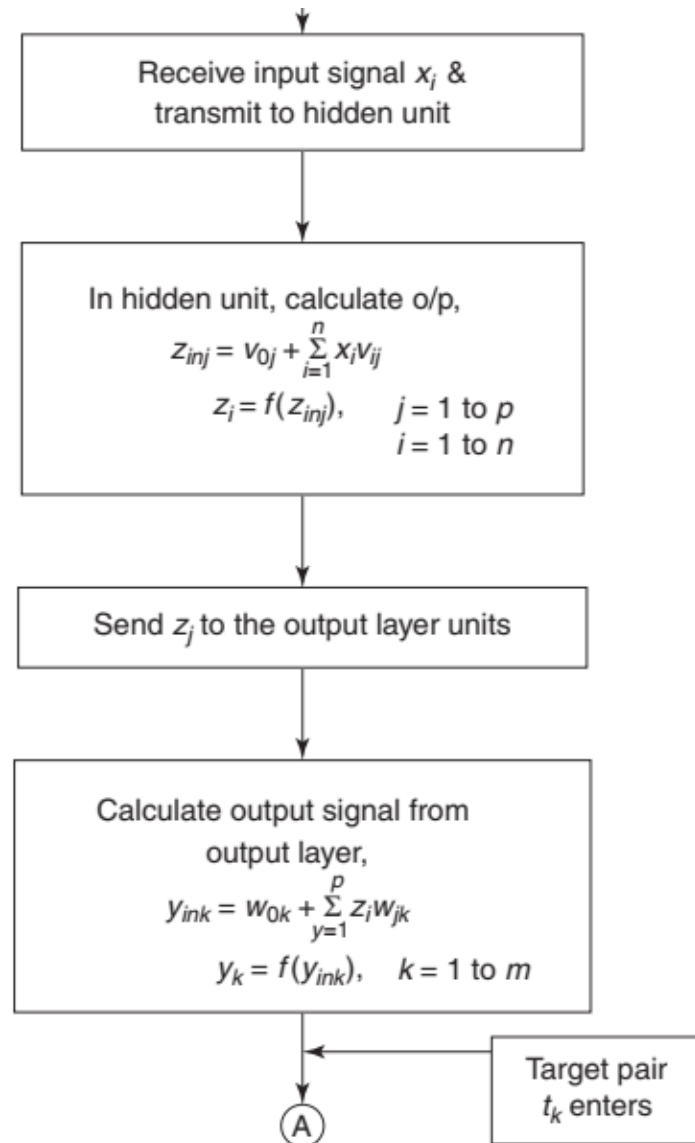
Each hidden unit (z_j , $j = 1$ to p) updates its bias and weights:

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \Delta v_{ij}$$

$$v_{0j}(\text{new}) = v_{0j}(\text{old}) + \Delta v_{0j}$$

Step 9: Check for the stopping condition. The stopping condition may be certain number of epochs reached or when the actual output equals the target output.

Flowchart for Training Process



- The above algorithm uses the incremental approach for updation of weights, i.e., the weights are being changed immediately after a training pattern is presented. There is another way of training called *batch-mode training*, where the weights are changed only after all the training patterns are presented. The effectiveness of two approaches depends on the problem, but batch-mode training requires additional local storage for each connection to maintain the immediate weight changes.

Testing Algorithm of Back-Propagation Network

Step 0: Initialize the weights. The weights are taken from the training algorithm.

Step 1: Perform Steps 2–4 for each input vector.

Step 2: Set the activation of input unit for x_i ($i = 1$ to n).

Step 3: Calculate the net input to hidden unit x and its output. For $j = 1$ to p ,

$$z_{inj} = v_{0j} + \sum_{i=1}^n x_i v_{ij}$$
$$z_j = f(z_{inj})$$

Step 4: Now compute the output of the output layer unit. For $k = 1$ to m ,

$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$
$$y_k = f(y_{ink})$$

Use sigmoidal activation functions for calculating the output.